

Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ

к лабораторной работе №3
на тему

**АСИММЕТРИЧНАЯ КИПТОГРАФИЯ.
КРИПТОСИСТЕМА РАБИНА**

БГУИР КП 6-05-0612-02 015 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф. Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение	3
1 Выполнение работы	4
Вывод	6
Приложение А (обязательное) Листинг программного кода	7

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается асимметричная криптосистема Рабина. В отличие от симметричных алгоритмов, в ней используются открытый и закрытый ключи. Открытый ключ содержит модуль n , являющийся произведением двух простых чисел p и q , а закрытый ключ содержит сами множители p и q .

Зашифрование в криптосистеме Рабина выполняется возведением сообщения в квадрат по модулю n :

$$c = m^2 \bmod n.$$

При расшифровании необходимо найти квадратные корни полученного значения по модулю n . В общем случае существует четыре возможных корня, поэтому практическая реализация должна обеспечивать выбор правильного варианта.

Целью лабораторной работы является разработка программного средства на языке C++, реализующего генерацию ключей, зашифрование и расшифрование файлов с использованием криптосистемы Рабина.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для работы с криптосистемой Рабина. Программа поддерживает генерацию ключевой пары, зашифрование файлов с использованием открытого ключа и расшифрование с использованием закрытого ключа.

Для работы с большими целыми числами используется библиотека Boost.Multiprecision и тип `cpp_int`. Генерация ключей выполняется функцией `generateKeyPair`. Размер модуля задаётся пользователем, при этом он должен быть чётным и не меньше 32 бит. Формируются два различных простых числа одинаковой разрядности, после чего вычисляется модуль

$$n = pq.$$

В реализации используются простые числа, удовлетворяющие условию $p \equiv 3 \pmod{4}$ и $q \equiv 3 \pmod{4}$. Для поиска таких чисел применяется вероятностный тест Миллера–Рабина. Полученные параметры сохраняются в отдельные файлы: открытый ключ содержит значение n , а закрытый – значения p и q .

Перед зашифрованием исходный файл разбивается на блоки длиной не более 27 байт. Каждый блок дополнительно содержит служебные данные: маркер, длину полезной части и контрольную сумму. Общий размер сформированной записи составляет 31 байт. Такая структура используется для проверки корректности найденного корня при расшифровании.

Полученная запись преобразуется в большое целое число m , после чего выполняется основное преобразование Рабина:

$$c = m^2 \pmod{n}.$$

В зашифрованный файл также записываются служебные данные, содержащие сигнатуру формата RABIN1, значение модуля, исходный размер файла и количество блоков. Значения шифртекста записываются последовательно, по одному на строку.

Расшифрование выполняется с использованием закрытых параметров p и q . Для каждого значения шифртекста сначала находятся квадратные корни по модулям p и q . Затем с помощью расширенного алгоритма Евклида и китайской теоремы об остатках формируются четыре возможных корня по модулю n .

Для каждого из четырёх вариантов выполняется проверка структуры блока. Проверяется наличие специального маркера, допустимость длины данных и совпадение контрольной суммы. Корень, прошедший эти проверки, считается корректным исходным блоком. После обработки всех блоков восстановленные данные записываются в выходной файл.

Для запуска программы предусмотрены три операции:

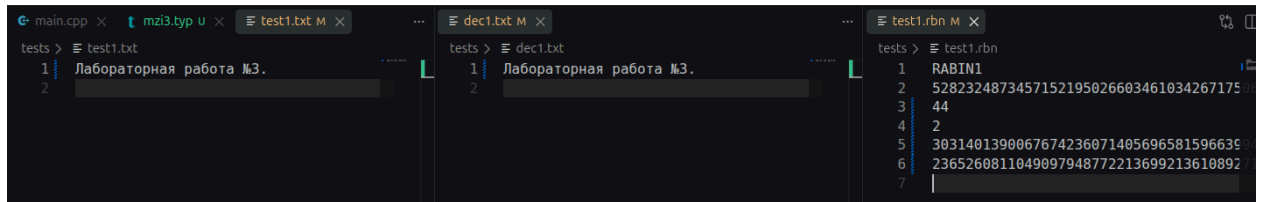
```
program keygen 256 public.key private.key
program enc secret.txt secret.rbn
public.key
program dec secret.rbn restored.txt private.key
```

Первая команда создаёт ключевую пару, вторая выполняет зашифрование файла, а третья – его расшифрование.

Главный модуль `main.cpp` отвечает за разбор аргументов командной строки, вызов соответствующих функций и обработку ошибок. Основные функции

криптосистемы объявлены в `rabin.hpp` и реализованы в отдельном исходном модуле.

Для проверки работы программы выполняется последовательное зашифрование и расшифрование тестового файла. После расшифрования содержимое восстановленного файла сравнивается с исходным.



```
main.cpp x mzi3.ty u x test1.txt M x ... dec1.txt M x ... test1.rbn M x
tests > test1.txt
1 Лабораторная работа №3.
2
tests > dec1.txt
1 Лабораторная работа №3.
2
tests > test1.rbn
1 RABIN1
2 5282324873457152195026603461034267175
3 44
4 2
5 3031401390067674236071405696581596639
6 2365260811049097948772213699213610892
7
```

Рисунок 1.1 – Вывод программы при генерации ключевой пары, зашифровании и расшифровании файла с использованием криптосистемы Рабина

В результате тестирования проверяется корректность генерации ключей, зашифрования и расшифрования данных. Совпадение исходного и восстановленного файлов подтверждает правильность работы реализованного программного средства.

Таким образом, в ходе работы была реализована криптосистема Рабина с поддержкой работы с файлами, большими целыми числами и несколькими возможными квадратными корнями при расшифровании.

ВЫВОД

В ходе выполнения лабораторной работы была изучена криптосистема Рабина и разработано программное средство на языке C++, реализующее генерацию ключевой пары, зашифрование и расшифрование файлов.

В программе реализовано формирование модуля n из двух простых чисел, удовлетворяющих условию $3 \bmod 4$, вычисление шифртекста посредством возведения сообщения в квадрат по модулю n и восстановление исходных данных по четырём возможным квадратным корням.

Для выбора правильного корня используется служебная структура блока и контрольная сумма. Проведённое тестирование показало, что после последовательного зашифрования и расшифрования исходное содержимое файла восстанавливается.

В результате выполнения лабораторной работы были получены практические навыки реализации асимметричного криптографического алгоритма Рабина, работы с большими целыми числами и организации файлового формата для хранения зашифрованных данных.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – main.cpp

```
#include "rabin.hpp"
#include <cstdlib>
#include <exception>
#include <iostream>
#include <string>

void printUsage(const char* programName)
{
    std::cerr
        << "Usage:\n"
        << " " << programName << " keygen <bits> <public-key> <private-
key>\n"
        << " " << programName << " enc <input> <output> <public-key>\n"
        << " " << programName << " dec <input> <output> <private-key>\n\n"
        << "Parameters:\n"
        << " keygen : Generate Rabin keys. <bits> is the modulus size.\n"
        << " enc    : Encrypt the input file with the public key.\n"
        << " dec    : Decrypt the input file with the private key.\n\n"
        << "Examples:\n"
        << " " << programName << " keygen 256 public.key private.key\n"
        << " " << programName << " enc secret.txt secret.rbn public.key\n"
        << " " << programName << " dec secret.rbn restored.txt
private.key\n";
}

int main(int argc, char* argv[])
{
    if (argc < 2) {
        printUsage(argv[0]);
        return 1;
    }

    const std::string command = argv[1];

    try {
        if (command == "keygen") {
            if (argc != 5) {
                printUsage(argv[0]);
                return 1;
            }

            const unsigned bits = static_cast<unsigned>(std::stoul(argv[2]));
            const std::string publicKeyFile = argv[3];
            const std::string privateKeyFile = argv[4];

            const RabinKeyPair keys = generateKeyPair(bits);

            if (!savePublicKey(publicKeyFile, keys.n)) {
                std::cerr << "Error: cannot write public key file.\n";
                return 1;
            }

            if (!savePrivateKey(privateKeyFile, keys.p, keys.q)) {
                std::cerr << "Error: cannot write private key file.\n";
                return 1;
            }
        }
    }
```

```

        std::cout << "Keys generated successfully.\n";
        return 0;
    }

    if (command == "enc") {
        if (argc != 5) {
            printUsage(argv[0]);
            return 1;
        }

        const bool success = encryptFile(
            argv[2],
            argv[3],
            argv[4]);

        if (!success) {
            std::cerr << "Error: encryption failed.\n";
            return 1;
        }

        std::cout << "Encryption completed successfully.\n";
        return 0;
    }

    if (command == "dec") {
        if (argc != 5) {
            printUsage(argv[0]);
            return 1;
        }

        const bool success = decryptFile(
            argv[2],
            argv[3],
            argv[4]);

        if (!success) {
            std::cerr << "Error: decryption failed.\n";
            return 1;
        }

        std::cout << "Decryption completed successfully.\n";
        return 0;
    }

    std::cerr << "Error: unknown command '" << command << "'.\n";
    printUsage(argv[0]);
    return 1;
}

catch (const std::exception& ex) {
    std::cerr << "Error: " << ex.what() << '\n';
    return 1;
}
}

```

Листинг А.2 – rabin.hpp

```

#pragma once

#include <boost/multiprecision/cpp_int.hpp>
#include <cstdint>
#include <string>

```

```

using boost::multiprecision::cpp_int;
using Byte = std::uint8_t;

struct RabinKeyPair {
    cpp_int p;
    cpp_int q;
    cpp_int n;
};

RabinKeyPair generateKeyPair(unsigned modulusBits);

bool savePublicKey(const std::string& fileName, const cpp_int& n);
bool savePrivateKey(const std::string& fileName, const cpp_int& p, const
cpp_int& q);
bool loadPublicKey(const std::string& fileName, cpp_int& n);
bool loadPrivateKey(const std::string& fileName, cpp_int& p, cpp_int& q);

bool encryptFile(const std::string& inputFile,
                 const std::string& outputFile,
                 const std::string& publicKeyFile);

bool decryptFile(const std::string& inputFile,
                 const std::string& outputFile,
                 const std::string& privateKeyFile);

```

Листинг А.3 – file_utils.cpp

```

#include "file_utils.hpp"

#include <fstream>

bool readBinaryFile(const std::string& fileName, std::vector<Byte>& data)
{
    std::ifstream file(fileName, std::ios::binary);
    if (!file) {
        return false;
    }

    file.seekg(0, std::ios::end);
    const std::streamoff size = file.tellg();
    if (size < 0) {
        return false;
    }
    file.seekg(0, std::ios::beg);

    data.resize(static_cast<std::size_t>(size));
    if (!data.empty()) {
        file.read(reinterpret_cast<char*>(data.data()),
                 static_cast<std::streamsize>(data.size()));
    }

    return static_cast<bool>(file) || file.eof();
}

bool writeBinaryFile(const std::string& fileName, const std::vector<Byte>&
data)
{
    std::ofstream file(fileName, std::ios::binary);
    if (!file) {
        return false;
    }
}

```

```
if (!data.empty()) {  
    file.write(reinterpret_cast<const char*>(data.data()),  
               static_cast<std::streamsize>(data.size()));  
}  
  
return static_cast<bool>(file);  
}
```